Exercise 5 - Churn Prediction

Train and compare three classifiers on the subscriber master file, then choose an operating point you can defend in rupees.

MODE	DURATION	PREREQUISITE	DELIVERABLE
Classroom	**75 minutes**	**Module 2 complete**	**Notebook + comparison table**

1. Objective

By the end of this exercise, you will have trained logistic regression, decision tree and random forest classifiers on identical splits of the telecom master file, compared them honestly, and selected a decision threshold from a stated business cost ratio rather than accepting the software default.

The exercise is only partly about the models. The columns you decide to exclude, and the reason you give for excluding them, carry more marks than the score you achieve.

2. Prerequisites

- Python 3.10 or later with pandas, scikit-learn, matplotlib and seaborn installed.
- Jupyter Notebook or JupyterLab, launched from the folder holding the dataset.
- The confusion matrix and the four-evaluation metrics covered in the Part 1 session.

3. Files required

File	Description
telecom_master.csv	3,000 subscribers, 22 columns, one row per subscriber
Module_4_Data_Dictionary.docx	Column-by-column description, including when each value is recorded

4. Procedure

Step 1 - Set up and load

Create a notebook named churn_model.ipynb in the same folder as the dataset.

```
import pandas as pd, numpy as np, matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import (confusion_matrix, classification_report,
                             roc_auc_score, precision_score, recall_score)

df = pd.read_csv('telecom_master.csv')
print(df.shape)
print(df['churn'].value_counts(normalize=True).round(3))
```

EXPECTED 3000 rows, 22 columns. The churn rate is a little under a quarter - note the exact figure, you will need it as your baseline.

Step 2 - The leakage audit

Before any modelling, work through every column and apply the day-before test: would this value exist, with this value, the day before the subscriber churned? Record your verdict in a table.

Column	Recorded when?	Verdict
tenure_months	Continuously, from activation	Keep
complaints_6m	Rolling window, always available	Keep
retention_call_flag	?	?
total_charges	?	?
...	Complete for all 22 columns	

Two columns in this file fail the test. One fails it for churn, and one fails it for the ARPU task in Exercise 6. Find them by reasoning about how the business generates each value - not by looking at correlations.

LEARNING POINT A correlation check will find the leak, but only after you have already been misled by the score. Reasoning about the data-generating process finds it first, and is the habit that transfers to datasets where you have no answer key.

Step 3 - Demonstrate the leak, then remove it and Add the feature Engineering

Feature engineering:

Right now you have complaints_6m (raw complaint count) and tenure_months sitting as two separate columns. But 2 complaints means very different things depending on context:

- A subscriber who joined **2 months ago** with 2 complaints — that's a red flag, something's going wrong early
- A subscriber who's been around **5 years** with 2 complaints — that's basically a non-event, everyone has an issue occasionally over 5 years

If you feed the model complaints_6m on its own, it treats those two subscribers as equally risky. The model can *sort of* work this out itself if given enough data and the right algorithm (a tree can learn to split on both columns together), but you can make the signal much sharper — and much easier for a linear model or a human to interpret — by doing the division yourself.

```
df['complaint_intensity'] = df['complaints_6m'] / (df['tenure_months'] / 6).clip(lower=1)
# then complaint_intensity joins num_cols the same way avg_monthly_gb etc. do
```

Train one quick random forest with the suspect column included, note the ROC-AUC, then retrain without it. Record both numbers - the gap is the evidence for your write-up.

```
target = 'churn'
leaky  = ['retention_call_flag', 'total_charges']   # replace with your own findings

def make_X(frame, drop):
    X = frame.drop(columns=[target] + drop + ['customer_id'])
    return X

num_cols = make_X(df, leaky).select_dtypes(include=np.number).columns.tolist()
cat_cols = make_X(df, leaky).select_dtypes(exclude=np.number).columns.tolist()
print(len(num_cols), 'numeric,', len(cat_cols), 'categorical')
print('complaint_intensity' in num_cols)   # should print True
```

Step 4 - Build a leak-free pipeline and split

Every transformation must be fitted on the training fold only. A ColumnTransformer inside a Pipeline guarantees this; doing it by hand on the full frame does not.

```
X = make_X(df, leaky)
y = df[target]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, stratify=y, random_state=42)

pre = ColumnTransformer([
    ('num', Pipeline([('imp', SimpleImputer(strategy='median')),
                      ('sc',  StandardScaler())]), num_cols),
    ('cat', Pipeline([('imp', SimpleImputer(strategy='most_frequent')),
                      ('oh',  OneHotEncoder(handle_unknown='ignore'))]), cat_cols),
])
```

LEARNING POINT stratify=y matters here. Without it, a 25% test split on a 23% positive class can hand you a test set with a materially different base rate, and every metric you report becomes hard to compare.

Step 5 - Train and compare three classifiers

```
models = {
    'Logistic regression': LogisticRegression(max_iter=2000),
    'Decision tree':       DecisionTreeClassifier(max_depth=6, random_state=42),
    'Random forest':       RandomForestClassifier(n_estimators=300,
                                                  min_samples_leaf=3, random_state=42),
}

results = []
for name, clf in models.items():
    pipe = Pipeline([('pre', pre), ('clf', clf)]).fit(X_train, y_train)
    proba = pipe.predict_proba(X_test)[:, 1]
    pred  = (proba >= 0.5).astype(int)
    results.append({
        'model':     name,
```

```
        'roc_auc':   roc_auc_score(y_test, proba),
        'precision': precision_score(y_test, pred, zero_division=0),
        'recall':    recall_score(y_test, pred),
        'accuracy':  (pred == y_test).mean(),
    })

comparison = pd.DataFrame(results).round(3)
comparison
```

EXPECTED All three land close together, with ROC-AUC in the mid-0.7s and recall at the default threshold well under 0.3. That is the honest result on this data - do not tune until Step 8 tells you what to tune for.

Step 6 - Read the confusion matrix as costs

```
best = Pipeline([('pre', pre), ('clf', models['Random forest'])]).fit(X_train, y_train)
proba = best.predict_proba(X_test)[:, 1]
cm = confusion_matrix(y_test, (proba >= 0.5).astype(int))
tn, fp, fn, tp = cm.ravel()
print(f'TN {tn}  FP {fp}  FN {fn}  TP {tp}')
print(classification_report(y_test, (proba >= 0.5).astype(int), digits=3))
```

Write one sentence for each cell stating what it means for the retention team, not for the model.

Step 7 - Compare against the do-nothing baseline

```
baseline_accuracy = 1 - y_test.mean()
print('Predict nobody churns  - accuracy:', round(baseline_accuracy, 3))
print('Random forest at 0.5   - accuracy:', round(((proba >= 0.5).astype(int) == y_test).mean(), 3))
```

LEARNING POINT If your model's accuracy is not clearly above the do-nothing baseline, accuracy is telling you nothing at all. This is the single most common way a churn model is reported as a success when it is not.

Step 8 - Sweep the threshold and choose one

```
rows = []
for t in np.arange(0.10, 0.65, 0.05):
    pred = (proba >= t).astype(int)
    rows.append({'threshold': round(t, 2),
                 'precision': precision_score(y_test, pred, zero_division=0),
                 'recall':    recall_score(y_test, pred),
                 'flagged':   int(pred.sum())})
sweep = pd.DataFrame(rows).round(3)

plt.plot(sweep.threshold, sweep.precision, marker='o', label='Precision')
plt.plot(sweep.threshold, sweep.recall,    marker='o', label='Recall')
```

```
plt.xlabel('Decision threshold'); plt.legend(); plt.grid(alpha=.3); plt.show()
sweep
```

Now state your cost ratio. Using the figures from the session - a false negative costs roughly ₹5,880 in lost lifetime value and a false positive costs roughly ₹300 in retention spend - the ratio is about 20 to 1. Choose the threshold that best serves that ratio and write down why.

```
COST_FN, COST_FP = 5880, 300
sweep['expected_cost'] = 0
for i, t in enumerate(sweep.threshold):
    pred = (proba >= t).astype(int)
    tn, fp, fn, tp = confusion_matrix(y_test, pred).ravel()
    sweep.loc[i, 'expected_cost'] = fn * COST_FN + fp * COST_FP
sweep.sort_values('expected_cost').head()
```

LEARNING POINT The threshold that minimises expected cost is almost never 0.5, and it moves when finance revises either figure. State the assumption alongside the number, every time.

5. Expected outcomes

- **Audit -** A completed leakage audit table covering all 22 columns, with a verdict and a reason for each.
- **Evidence -** Recorded ROC-AUC with and without the leaking column, and one sentence explaining the gap.
- **Comparison -** A three-row comparison table of the classifiers on identical splits.
- **Decision -** A threshold sweep, a chosen threshold, and a written justification naming your cost ratio.

6. Good practices

- Set random_state everywhere. A result that changes between runs cannot be discussed.
- Keep every transformation inside the pipeline. If you find yourself calling fit on the full frame, stop.
- Report the metric that answers the question being asked, and say which one you chose and why.
- Never delete a column silently. Every exclusion needs a line in the audit table.
- If a score surprises you, treat it as a defect report against your own pipeline until proven otherwise.